RUDP Project: The Owner's Manual

Computer Networks project: Harshavardhan Mallela, Rohit Arya, Shloak Sinha. 9 Oct 2026.

We built a delivery service that keeps UDP's speed but adds TCP's reliability, taught it to adjust its own speed, tested it in a simulated four-router campus network and watched it work packet by packet in Wireshark. This manual explains each part the way a car manual explains the engine, the cruise control and the dashboard.

The car at a glance

Six parts, each with a car equivalent. The code lives in the GitHub repo Capsturrrr/rudp.

Part	What it does	Car equivalent	Where
RUDP	Sends data over UDP and makes sure every piece arrives, in order	The engine	`src/client.c`, `src/server.c`, `src/common.c`
Congestion control (AIMD)	Speeds up while the road is clear, slows down on loss	Automatic gearbox	inside `client.c`
Smart-RUDP	Learns the best speed with Q-learning instead of a fixed rule	Adaptive cruise control	`src/smart client.c`, `src/rl_cc.h`
RUDP Chat	A messenger on RUDP packets: terminal and browser versions, works between two devices, loss slider	The test drive	`src/chat.c`, `web/rudp web.py`, `web/index.html`
Packet Tracer network	Four routers, two VLANs, DHCP, an access list, failover	The road	`network/cfg/`, the `.pkt` file
Wireshark dissector	Decodes our custom packets so you can read them	The dashboard	`network/rudp.lua`, the `.pcapng` files

In one line: the engine moves data, the cruise control tunes its speed, the road is the network it travels over, and Wireshark is how we see what happened.

Start the engine

Run these in WSL, once, before any demo:

```
cd ~/projects/rudp
git pull
make all smart chat
```

RUDP with 20% packet loss. Open two terminals.

```
./bin/server        # terminal 1
./bin/client 20     # terminal 2
```

Look for "All 62 packets delivered" and the reassembled message on the server.

RUDP Chat. Start both before typing. The last number is the loss percentage.

```
./bin/chat 9001 9002 Alice 30   # terminal 1
./bin/chat 9002 9001 Bob 30     # terminal 2
```

RUDP Web Chat (browser). Same protocol, shown in a web page. Two copies on one laptop:

```
python3 web/rudp_web.py --udp 9001 --peer 127.0.0.1:9002 --name A --http 8080
python3 web/rudp_web.py --udp 9002 --peer 127.0.0.1:9001 --name B --http 8081
```

Open `http://localhost:8080` and `http://localhost:8081` side by side. Use `http://`, not `https://`.

Smart-RUDP vs AIMD. One run on the lossy path; compare the completion time and retransmissions.

```
gcc -DSERVER_PORT=9301 -o bin/server_9301 src/server.c src/common.c
./bin/server_9301 > /dev/null 2>&1 &
./bin/smart_client --mode aimd --port 9301 --delay 25 --jitter 10 --loss 2 --rate 800 --q
ueue 50 --scenario lossy --packets 1200 --qfile results/q_lossy.txt --seed 1001 | grep RE
SULT
./bin/smart_client --mode rl --port 9301 --delay 25 --jitter 10 --loss 2 --rate 800 --que
ue 50 --scenario lossy --packets 1200 --qfile results/q_lossy.txt --seed 1001 | grep RESU
LT
pkill server_9301
```

The full step-by-step demo script is in the file Demo_Steps.txt and in `docs/DEMO_RUNBOOK.md` in the repo.

The engine: RUDP

UDP is like dropping letters in a postbox: fast, but a letter can get lost, arrive twice or arrive out of order, and nobody tells you. RUDP adds the checks TCP has, written by us in C.

The packet. Every packet starts with a 13-byte header: sequence number (4 bytes, which packet this is), acknowledgment number (4, the next packet I expect), flags (1, what kind of packet), checksum (2, detects corruption) and payload length (2). Data follows. Flags: SYN starts a connection, ACK confirms, FIN ends it, DATA carries data.

The handshake. Client sends SYN, server answers SYN+ACK, client sends ACK. Only then does data flow. This is the same three-way handshake TCP uses.

Reliability. Each data packet has a number. The receiver accepts only the next expected number and replies with a cumulative ACK ("I have everything up to here"). Anything out of order is thrown away. The sender keeps a window of packets in flight (Go-Back-N). If no ACK arrives within 500 ms, it resends everything from the first unacknowledged packet. If three duplicate ACKs arrive, it resends right away (fast retransmit).

Congestion control. The window size decides how fast we send. It starts at 1 and doubles each round trip (slow start) until a threshold of 16, then grows by about 1 per round trip (congestion avoidance). On a loss it is cut: halved on three duplicate ACKs, reset to 1 on a timeout. This is AIMD, additive increase and multiplicative decrease.

Loss simulation. The client can drop packets on purpose (`./bin/client 20` drops 20%), so you can see the recovery working.

Cruise control that learns: Smart-RUDP

AIMD follows one fixed rule whatever the road looks like. Smart-RUDP replaces that rule with a small learning agent (tabular Q-learning) that picks how to change the window.

How it works. Every round trip the agent looks at three things: how much the delay has risen (RTT ratio), how much loss there was, and the current window size. That gives 80 situations (states). For each it chooses one of five actions: multiply the window by 0.5, 0.85, 1.0, 1.15 or 1.5. It is rewarded for throughput and punished for delay and loss, and it updates a table of scores as it goes (learning rate 0.15, discount 0.9).

What stays the same. Both modes share all the reliability code (Go-Back-N, adaptive timeout, fast retransmit). Only the window policy differs, so the comparison is fair.

How it was tested. Three emulated paths, each trained for 40 episodes, then 6 evaluation runs each: 5G-like (low delay), lossy wireless (2% loss) and satellite-like (150 ms delay). Loss, delay and a bottleneck queue are emulated inside the client.

The result, honestly. The first learner, a table of 80 states, matched AIMD on the lossy path and was slower on the other two; a simple table does not beat a well-tuned rule. The small neural network did, and in the final system (Selective Repeat, window cap 128, slow start, pacing, hand-over to AIMD under heavy loss) it needs 15 to 52 percent less time than a tuned AIMD on four ordinary paths, ties on a 10 percent loss path, and takes more than its share of a shared link. Say all of this plainly if asked.

Run it. See "Start the engine" above. Graphs are in `smart_graphs/`, raw results in `results/`.

The test drive: RUDP Chat

A messenger built on the same packet format, so the protocol is something you can see working. Each message is one numbered packet. The receiver shows messages in order and sends ACKs; the sender resends anything lost. It comes in three forms.

1. Terminal, one machine. Two terminals, ports swapped, last number = percent of packets the simulated network drops.

```
./bin/chat 9001 9002 Alice 30
./bin/chat 9002 9001 Bob 30
```

Lines in brackets show the network losing a message and RUDP recovering: "network lost message #3", "timeout: resending", "3 duplicate ACKs: fast retransmit". Every message still arrives complete and in order. Ctrl+D quits and prints totals.

2. Terminal, two machines. The peer is written as ip:port. Both sides start within a few seconds of each other. Each side sends small packets to the other so home routers open a return path (NAT hole punching), then prints "Connected to ...".

```
./bin/chat 9001 FRIENDS_IP:9001 Harsh 20
```

Hostel Wi-Fi and mobile data often block direct connections. We used Tailscale (a private network): install it on both devices, sign in, and use the `100.x.x.x` address each device shows. Our tailnet: laptop `100.74.161.24`, phone `100.88.28.25`. On the phone the programs run in Termux. WSL sits behind Windows, so if the laptop side does not connect, install Tailscale inside WSL too and use that address.

3. Browser version. Browsers cannot send raw UDP, so `web/rudp_web.py` (Python, no installs) speaks RUDP to the peer and serves the page on localhost. It talks to the C chat as well.

```
python3 web/rudp_web.py --udp 9001 --peer 100.88.28.25:9001 --name Harsh --http 8080
```

Open `http://localhost:8080`. The page shows a delivered tick on each message once it is acknowledged, a packet-loss slider, counters (sent, lost, resent), how many messages are in flight, and a log of timeouts and fast retransmits. In Termux, add `> /dev/null 2>&1 &` to the end of the command so the prompt stays free; stop it with `pkill -f rudp_web`.

Window controller switch (browser version). The dropdown on the right chooses fixed window (8), AIMD, or Smart-RUDP. Smart-RUDP loads `results/q_lossy.txt` and keeps learning while you chat. "Send 60 test messages" times the delivery. Start both web copies with `--loss 10 --delay 40` to emulate a slow lossy path. Smart-RUDP for the chat was retrained on simulated paths (`python3 web/train.py`, writes `results/q_chat.txt`). On six held-out simulated paths it was faster than AIMD on five and equal on one (Table 11 in the report). A stress test on 48 random unseen paths (`python3 web/stress.py`) gave mean delivery times of 3.73 s for Smart-RUDP, 3.88 s for the fixed window and 7.16 s for AIMD; Smart-RUDP was faster than AIMD on 27 paths, slower on 5 and within 5% on 16. It is weakest on tight bottleneck queues and very high loss. The dropdown also has a deep (neural network) option: `web/dqn.py`, trained with `python3 web/train_deep.py` (writes `results/dqn_chat.json`, picks the best of four seeds on validation paths because one seed in four fails). On the same 48 paths it averaged 3.20 s; it runs only in the Python gateway, not in the C code. AIMD loses because AIMD resets its window to one packet after every timeout even when loss is random. Both start from the same window of 10. The agent may not shrink the window without a loss or delay signal; without that rule it lost. In the earlier experiments (Section 8 of the report) it did not beat AIMD, and that result stands.

What it proves. At 30% loss in both directions, messages from the browser and from the terminal peer all arrived once and in order in our tests.

Limits. No accounts or encryption. Loss is simulated inside the program. The terminal version uses a fixed window of 8; only the browser version can use Smart-RUDP. Over the internet the result depends on the network between the peers.

The road: the Packet Tracer network

Four routers run OSPF (a protocol where routers share what they know and pick the shortest path). A student PC on VLAN 10 reaches the server across the core. In Packet Tracer the routers are labelled Router1, Router2, Router3 and Router0, configured as R1, R2, R3 and R4.

Link	Subnet	Notes
R1 Gi0/1 to R2 Gi0/0	10.0.12.0/30	R1 .1, R2 .2
R2 Gi0/1 to R3 Gi0/1	10.0.23.0/30	R2 .1, R3 .2
R1 Gi0/2 to R3 Gi0/2	10.0.13.0/30	primary path to the server
R2 Gi0/2 to R4 Gi0/0	10.0.24.0/30	R2 .1, R4 .2
VLAN 10 students	192.168.10.0/24	gateway R1 .1
VLAN 20 staff	192.168.20.0/24	gateway R1 .1
Server LAN	192.168.30.0/24	gateway R3 .1, server 192.168.30.10
Branch LAN	192.168.40.0/24	gateway R4 .1

What each feature does. VLANs split one switch into separate student and staff networks. Router-on-a-stick lets R1 route between them over one cable. DHCP hands out addresses automatically. The access list on R1 blocks students from reaching staff. OSPF picks routes: server LAN is metric 2 via R3, branch LAN is metric 3 via R2.

What we tested. R2 shows three FULL OSPF neighbours. PC0 traces to the server through 192.168.10.1, 10.0.13.2, then the server. The branch PC reaches the server. Pinging the staff PC from the student PC fails (access list), while the server still answers. When R1's link to R3 is shut down mid-ping, 70 of 71 pings still succeed and the route switches to R2 and R3.

Good to know. The cabling of R2 did not match the first plan. We found the real wiring with the command `show cdp neighbors` and changed R2's addresses to match.

Limit. Packet Tracer cannot run our C programs. The network design and the protocol are validated separately, and the protocol was tested on one machine with emulated paths.

The dashboard: Wireshark

Wireshark shows the actual packets on the wire. It does not know our custom protocol, so we wrote a small plug-in, `network/rudp.lua`, that decodes our 13-byte header into readable fields. (Wireshark already has a different built-in protocol called RUDP, so ours is named `smartrudp` internally; the packet list still says RUDP.)

Files. `rudp_capture_loss.pcapng` is a real capture of our client and server with packet loss, including the handshake. `smart_lossy_aimd.pcap` and `smart_lossy_rl.pcap` are traces exported by our own client for the AIMD and Smart-RUDP runs.

Open it (from WSL; Windows Wireshark cannot see the WSL network, so we open a saved file):

```
"/mnt/c/Program Files/Wireshark/Wireshark.exe" -X 'lua_script:C:\Users\Hanuman\Downloads
\rudp_wireshark\rudp.lua' 'C:\Users\Hanuman\Downloads\rudp_wireshark\rudp_capture_loss.pc
apng'
```

Two filters to show. Type in the bar at the top and press Enter.

- `frame.number <= 3` shows the handshake: SYN, SYN-ACK, ACK. Click packet 1 and expand RUDP to see sequence number, ack, flags, checksum and length.
- `smartrudp.seq == 1327198699` shows one data segment twice, at 0.0004 s and again at 0.501 s. That 500 ms gap is the timeout and Go-Back-N resend.

Other useful filters: `smartrudp.flags.data == 1` (data only) and `smartrudp.flags.ack == 1` (ACKs only; repeated ack numbers are duplicate ACKs).

Warning lights

Symptom	Likely cause	Fix
`make` says smart or chat target not found	Old copy of the repo	`cd ~/projects/rudp && git pull`
Server prints nothing when output goes to a file	Output is buffered	Run it in a terminal, or use `stdbuf -oL ./bin/server`
Chat shows no messages	The other terminal was not started, or the ports are not swapped	Start both; first port is yours, second is the peer's
Smart-RUDP run hangs or errors on bind	A leftover server still holds the port	`pkill server_9301` and run again
Wireshark error "two protocols with the same name"	Old `rudp.lua` loaded twice	Delete any `rudp.lua` in `%APPDATA%\Wireshark\plugins` and use the command above
Packets show as plain UDP in Wireshark	Dissector not loaded	Analyze, Decode As, UDP port 8888 = RUDP
Packet Tracer interface up but protocol down	The other end is not configured yet	Configure it, wait 30 to 40 seconds of simulated time
`show ip ospf neighbor` is empty	OSPF has not converged, or an address does not match the cable	Fast Forward Time; `show cdp neighbors` to check the real wiring
Ping to the server is slow after the failover test	R1 Gi0/2 is still shut down	`interface gigabitEthernet0/2`, then `no shutdown`
First ping of a run times out	ARP is resolving	Normal; run it again
Browser says "This site can't provide a secure connection"	Chrome tried https	Type the whole address with `http://`
Termux prompt never comes back	The web server runs in the foreground	Ctrl+C (CTRL on the extra keys, then c), or add `&` to run it in the background
`web/rudp_web.py` not found on the phone	The clone is older than the web folder	`cd ~/rudp && git pull`

Symptom	Likely cause	Fix
Web page stays amber (waiting for peer)	Other side not started, wrong address, or Tailscale off	Start both within seconds; check the 100.x addresses; check the Tailscale app

Why this is a project, not a lab exercise

A lab exercise follows a given specification. A project has a problem, a design, evidence and failures to handle. This one has:

- **A problem we chose.** UDP is fast but unreliable, TCP is reliable but heavy: what does it take to build TCP's guarantees on UDP?
- **A multi-part design.** A 13-byte header, handshake, Go-Back-N, cumulative ACKs, fast retransmit, AIMD congestion control and loss simulation. A UDP lab stops at sending datagrams.
- **Measured evidence.** Benchmark sweep, congestion-window graphs, real loss captured in Wireshark with our own dissector, and an honest Smart-RUDP vs AIMD comparison.
- **Documented problems and fixes.** The report has a section on what broke and how it was fixed.
- **Integration.** The network design (OSPF, VLANs, ACL, failover) and a working chat between two devices.
- **A negative result reported honestly.** Smart-RUDP did not beat AIMD.

One line: a lab exercise sends a datagram; we built the reliability and congestion control that sit on top of it, measured them, and used them in a real application.

Questions you will be asked

How is this different from a normal UDP lab program? A lab program just sends datagrams. Ours adds sequence numbers, checksums, acknowledgments, retransmission, a sliding window, a handshake and congestion control, which is what TCP gives, built by us on top of UDP.

Why not just use TCP? TCP is inside the operating system and cannot be changed. Writing our own makes every mechanism visible and tunable. QUIC and HTTP/3 do the same in practice, to avoid TCP's head-of-line blocking and slow handshakes.

Did Smart-RUDP beat AIMD? In the final system, yes against a tuned AIMD: 15 percent on the 5G-like path, 49 on lossy, 52 on satellite-like and 52 on the fast path (real clock, 10 paired runs; the same ordering on real sockets through a proxy and on 100 random held-out paths in virtual time). On a path with 10 percent random loss it ties. It also beats tuned CUBIC on lossy and satellite-like paths. The first tabular agent did not beat AIMD, and earlier larger margins were partly caused by a timer defect that hurt AIMD more; it is fixed.

Does the protocol run inside Packet Tracer? No. Packet Tracer cannot run C programs. The network design is validated in Packet Tracer; the protocol is validated on one machine with emulated loss and delay, plus Wireshark captures. The report says this.

Are the protocol tests real-world? They show behaviour with emulated delay, loss and jitter on one machine. They do not prove internet performance, and the report says so. The chat between laptop and phone is a small real-network check.

Is the loss real? In the experiments and the chat it is simulated inside the program. We also captured a real loopback run with tshark and loss applied, which is the Wireshark capture.

Why does the exported trace have no handshake? The client's `--pcap` export builds packets with synthetic headers and skips the handshake. The real loopback capture taken with tshark shows SYN, SYN-ACK and ACK. Know which one you are showing.

What is the purpose? A readable, tunable reliable transport for campus users sending data to a server across a routed network, tested against routing, access policy and link failover.

Why Go-Back-N and not selective acknowledgments? We started with Go-Back-N because it is simple, then replaced it as the default with Selective Repeat (receiver buffers 256 packets and reports them in a 32-byte bitmap; sender has one timer per packet). Go-Back-N is still available for comparison.

What happens if a router link fails? OSPF reroutes. In our test, R1's link to R3 was shut down during a ping: 70 of 71 pings succeeded, and the route moved to the longer path through R2 and R3.

What happens if the peer disappears? The sender keeps retransmitting on timeout. A connection abort after repeated timeouts is not implemented and is the fix we would add. Say this plainly if asked.

Does the checksum catch errors? It is a 16-bit one's-complement sum over header and payload, like UDP and TCP. A corrupted packet fails the check and is dropped, then recovered by retransmission. Like any checksum it cannot catch every error pattern.

Is the Q-learning really learning? Yes: 80 states, 5 actions, 40 training episodes, a reward that balances throughput against delay and retransmissions. The learning-curve figure shows it. The agent is small and tabular with limited training.

Why a window cap of 32 and ssthresh 16? The cap stops a lossless run growing without bound. Below ssthresh the window doubles each round trip (slow start); above it grows by one (avoidance). On a timeout it resets to 1; on three duplicate ACKs it halves.

Who did what? Agree your answer as a team before the review and give the same account.

Name the limits before she does. No kernel netem or multi-machine test (script provided, no root here); the agent is not a polite neighbour (68 percent of a shared link against tuned AIMD) and TCP coexistence was not tested; no gain at 10 percent random loss; trained only in a simulator. Then say what you would do next: fairness-aware training, real traces, kernel tests.

The slides, in order

#	Slide	One thing to say
1	Title	Reliable UDP in C, plus a learning congestion controller

#	Slide	One thing to say
2	The problem	UDP is unreliable, TCP is closed; can we rebuild TCP's ideas?
3	Why we built it	Campus uploads to a server; QUIC does this in practice
4	What we built	Handshake, reliability, flow control, congestion control
5	Packet header	13 bytes, packed byte by byte
6	Handshake and data flow	SYN, SYN-ACK, ACK, data, ACKs, FIN
7	Go-Back-N	500 ms timer plus fast retransmit
8	AIMD	Slow start, avoidance, halve on loss
9	How we tested	Built-in loss, netem, Wireshark
10	Benchmark	62 of 62 delivered at every loss level
11	Problems fixed	Stuck server, netem, Wireshark and WSL
12	Literature	Jacobson, Chiu and Jain, RFC 5681, CUBIC, QUIC
13	Network design	Four routers, OSPF, VLANs, ACL
14	Packet Tracer results	Routes, 70/71 failover, ACL block
15	Smart-RUDP design	Q-learning, 80 states, 5 actions
16	Smart-RUDP result	Honest: tied on lossy, slower elsewhere
17	Wireshark traces	Dissector and retransmission counts
18	Wireshark real capture	Handshake and a repeated sequence number
19	RUDP Chat demo	The protocol in use, with a loss slider
20	Learnings and future work	Selective Repeat, better agent, real network
21	Thank you	GitHub link

Glossary

Term	Plain meaning
UDP	Sends packets with no guarantee they arrive
TCP	Reliable delivery built into the operating system
ACK	A receipt saying a packet arrived
Sequence number	The number of a packet, so order can be checked
Checksum	A small number that detects a damaged packet
Handshake	Three packets (SYN, SYN-ACK, ACK) that open a connection
Window	How many packets may be in flight without an ACK
Go-Back-N	On a loss, resend everything from the first lost packet
Fast retransmit	Resend at once after three duplicate ACKs, without waiting for a timeout
RTT	Round-trip time: how long a packet and its ACK take
Congestion control	Slowing down when the network is overloaded
AIMD	Grow the window slowly, cut it sharply on loss

Term	Plain meaning
Slow start	Doubling the window each round trip at the beginning
Q-learning	A learner that scores each action in each situation and improves with rewards
OSPF	A protocol where routers share links and pick the shortest path
VLAN	A separate virtual network on one switch
DHCP	Hands out IP addresses automatically
ACL	Access list: a router rule that permits or blocks traffic
Packet Tracer	Cisco's network simulator
Wireshark	A tool that shows packets in detail
Dissector	A plug-in that teaches Wireshark to read a protocol
pcap	A file of captured packets
netem	Linux tool that adds delay and loss to a network interface

Extra experiments (all simulated unless noted)

Run from the project folder. Each prints a table; none needs root.

Command	What it shows
`python3 web/stress.py`	48 unseen random paths: fixed window, AIMD, Smart-RUDP table, Smart-RUDP deep
`python3 web/baselines.py`	Adds plain UDP (68% delivered) and a TCP model (selective ACK). The TCP row is a model, not kernel TCP
`python3 web/sack_test.py`	Go-Back-N vs selective acknowledgment: AIMD mean 7.16 s to 4.90 s
`python3 web/reorder_test.py`	Reordering: AIMD with Go-Back-N goes from 0.82 s to 6.79 s at 20 ms jitter
`python3 web/train_deep.py`	Retrains the neural agent (four seeds, picks the best on validation paths)
`bash eval ci.sh 20 then python3 ci_summary.py`	20 runs per mode of the C experiments with 95% confidence intervals (takes about 30 minutes)
`sudo bash netem_baselines.sh lossy`	Real kernel TCP, plain UDP and RUDP on a netem path. Needs Linux with iproute2 (not run in our build environment); also `5g`, `sat`, `reorder`

Results with 20 runs: Smart-RUDP (C, table) is slower than AIMD on the 5G-like path (+0.92 s) and satellite-like path (+6.3 s), and ties on the lossy path (-0.37 s, not significant).

Neural agent and selective ACK in the C transport

```
make smart sack                     # bin/smart_client, bin/server, bin/server_sack
python3 web/export_dqn.py           # results/dqn_chat.json -> results/dqn_chat.txt (wei
ghts for C)
./bin/smart_client --mode deep --init 10 --dfile results/dqn_chat.txt --delay 25 --jitter
10 --loss 2 --rate 800 --queue 50 --packets 1200
./bin/smart_client --mode rl2  --init 10 --qfile results/q_chat.txt   ...same path option
s...
./bin/smart_client --mode aimd --init 10 ...                          # baseline with the
same start window
./bin/server_sack &  ./bin/smart_client --sack 1 --reorder 1 ...      # selective ACK, wi
th reordering on the path
make test                           # wire format, end to end, every mode, SACK, simulat
or
bash eval_ci2.sh 15 ; python3 ci_summary2.py     # AIMD vs table vs neural, 95% intervals
bash eval_ci3.sh 8 ; bash eval_ci3b.sh 8 ; python3 ci_summary3.py    # selective ACK arms
```

Results (C transport, emulated paths): the neural agent beats AIMD on all three paths (15%, 27%, 24%). The retrained table does not generalise (slower on 5G-like and satellite-like). Selective ACK cuts retransmissions about tenfold and does not change completion time significantly on these paths.

Fairness, ablations and the chat dashboard

```
python3 web/fairness_test.py     # two flows, one bottleneck: share and Jain index (resul
ts/fairness.txt)
python3 web/ablate.py            # retrains the neural agent with one piece removed (abou
t 10 min, results/ablation.txt)
python3 web/rudp_web.py --udp 9001 --peer IP:9001 --name Me --rto adaptive   # RFC 6298 t
imer; live dashboard is in the page
```

Findings: the neural agent is fair against itself and on short paths, but takes 74% against AIMD on the long 120 ms path. Only the no-shrink mask matters in the ablation (6.54 s without it, about 3.3 s with it, AIMD 7.16 s). The same training recipe varies 3.2 to 7.4 s across seeds. The chat gateway had a duplicate-ACK retransmission storm; it is fixed (a new fast retransmit is allowed only after the data outstanding at the last one is acknowledged).

Validity check (equal caps, realistic simulator)

```
RUDP_RG=1 python3 web/stress.py                    # simulator with the realistic fast-re
transmit rule
bash eval_ci4.sh 10 ; python3 ci_summary4.py        # C transport, AIMD cap 128 vs 32 vs
neural (original and retrained)
```

Result: with the realistic rule the simulated advantage of the neural agent mostly disappears (AIMD 7.75 s, neural 7.27 s). In C with equal caps the neural agent still wins by 19%, 29% and 28%. Quote the C numbers, not the simulator stress mean.

Hybrid safety net

```
./bin/smart_client --mode deep --init 10 --dfile results/dqn_chat.txt --hybrid 0.10 ...
# C transport
python3 web/rudp_web.py ... --hybrid 0.10
# chat gateway
bash eval_ci6.sh 10 5 ; python3 ci_summary6.py
# evidence (C, 4 scenarios)
RUDP_RG=1 RUDP_INORDER=1 RUDP_ARTO=1 python3 web/hybrid_eval.py 300
# calibrated simulator
```

Without it the agent is 7x slower than AIMD at 10% loss; with it the hostile path is back near AIMD and the three benign paths keep the agent's advantage.

Selective Repeat (default) and Go-Back-N

```
make smart gbn                                  # builds bin/server (Selective Repeat),
bin/server_gbn, bin/smart_client
./bin/smart_client --mode deep ...              # Selective Repeat (default)
./bin/smart_client --gbn --mode deep ...        # Go-Back-N (pair with bin/server_gbn)
RUDP_GBN=1 ./bin/client ; RUDP_GBN=1 ./bin/chat 9001 9002 Me 20    # original client / te
rminal chat in Go-Back-N
python3 web/rudp_web.py ... --gbn               # web chat in Go-Back-N
bash eval_ci7.sh 10 5 ; python3 ci_summary7.py  # evidence with Selective Repeat
```

Older result files (eval_ci*, except eval_ci3 and eval_ci7) were measured with Go-Back-N; their scripts now pin it explicitly.

Performance defaults and how to compare

```
./bin/smart_client --mode deep --dfile results/dqn_chat.txt ...                    # new
default: cap 128 + hand-over 0.10
./bin/smart_client --mode deep --maxcwnd 32 --hybrid 0 ...                         # the
original agent settings (or RUDP_CLASSIC=1)
./bin/smart_client --mode aimd --maxcwnd 128 --ssthresh 128 ...                    # tun
ed AIMD baseline
./bin/smart_client --mode cubic --maxcwnd 128 --ssthresh 128 ...                   # CUB
IC
bash eval_perf.sh 5 ; bash eval_perf2.sh 5 ; bash eval_perf3.sh 10 5 ; python3 ci_summary
_perf3.py
```

Final round: what changed and how to check it

```
make test ; make fuzz                          # now includes handshake regression, file
integrity, chat/gateway interop, CRC fuzzing
./filetransfer_demo.sh 2 lossy                 # real file over an impaired path, hashes
compared
bash eval_final.sh 10 5 ; python3 ci_summary_final.py    # final comparison (real clock):
aimd_32, aimd_128, cubic_128, deep
bash eval_proxy.sh 8 8 ; python3 ci_summary_proxy.py     # the same on real UDP sockets t
hrough bin/impair
bash eval_fair.sh 6                                      # two flows on one bottleneck, J
ain index
./bin/smart_client --vt 1 --mode deep ...                # virtual time: same sender, sim
ulated receiver and clock
python3 web/vt_named.py ; python3 web/vt_random.py 100 ; python3 web/vt_ablate.py ; pytho
n3 web/vt_calib.py
```

Defects the real-socket testing found: a lost final handshake ACK hung the connection (fixed as in TCP), spurious retransmission timeouts that hurt the baseline more than the agent (timer floor 1.3 x srtt), and a byte-sum checksum that let 2 percent of corrupted packets through (now CRC-16). Details: report Section 7.8 to 7.11 and 8.11 to 8.14.